Exp no - 12b
Date - 12/09/2026
Name - Andryan Durai Jebin

IMPLEMENT A CHAT PROGRAM USING SOCKET PROGRAMMING

AIM:

To implement a chat program using socket programming and enable communication between a client and a server.

ALGORITHM:

1. Start the server program.
2. Create a TCP socket.
3. Bind the socket to an IP address and port number.
4. Listen for incoming client connections.
5. Accept a connection request from the client.
6. Start the client program and connect it to the server.
7. Allow the client to enter and send messages to the server.
8. Receive and display the client's message on the server.
9. Allow the server to enter a reply and send it to the client.
10. Receive and display the server's reply on the client.
11. Repeat the communication until either side enters "exit".
12. Close the sockets and terminate the programs.

PROGRAMS:

SERVER PROGRAM (server.py)

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5001))
server.listen(1)

print("Chat Server is waiting for a client...")

conn, addr = server.accept()
print("Client connected:", addr)

while True:
 message = conn.recv(1024).decode()
```

```
if not message or message.lower() == "exit":
 print("Client disconnected.")
 break

print("Client:", message)

reply = input("Server: ")
 conn.sendall(reply.encode())

if reply.lower() == "exit":
 break

conn.close()
server.close()
```

CLIENT PROGRAM (client.py)

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5001))

print("Connected to Chat Server.")

while True:
 message = input("Client: ")
 client.sendall(message.encode())

if message.lower() == "exit":
 break

reply = client.recv(1024).decode()

if not reply:
 break

print("Server:", reply)

if reply.lower() == "exit":
 break

client.close()
```

PROCEDURE:

1. Save the programs as server.py and client.py.
2. Open two terminals in the same directory.
3. Run the server using the command: python server.py
4. Run the client in the second terminal using the command: python client.py
5. Enter a message in the client terminal.
6. View the received message in the server terminal.
7. Enter a reply in the server terminal and view it in the client terminal.
8. Continue exchanging messages and enter "exit" to terminate the chat.

OUTPUT:

Server Terminal:

Chat Server is waiting for a client...
Client connected: ('127.0.0.1', 54321)
Client: Hello!
Server: Hi! How are you?
Client: I am fine.
Server: Good to hear.

Client Terminal:

Connected to Chat Server.
Client: Hello!
Server: Hi! How are you?
Client: I am fine.
Server: Good to hear.

RESULT:

Thus, the chat program was implemented successfully using TCP socket programming, enabling messages to be exchanged between the client and server.